Lambda Calculus and Types miniproject

Jan Galek

Hilary Term 2026

Question 1

(a) We have $\lceil 0 \rceil \equiv \lambda sz.\, z$ and $\lceil n+1 \rceil \equiv \mathrm{nf}\big(\lambda sz.\, s\,\lceil n \rceil\big(\lceil n \rceil\, s\, z\big)\big)$, where $\mathrm{nf}(-)$ denotes β-normal form. Expanding the first few terms:

$$\lceil 1 \rceil = \lambda sz.\, s\lceil 0 \rceil z,$$

$$\lceil 2 \rceil = \lambda sz.\, s\lceil 1 \rceil(s\lceil 0 \rceil z),$$

$$\lceil 3 \rceil = \lambda sz.\, s\lceil 2 \rceil(s\lceil 1 \rceil(s\lceil 0 \rceil z)).$$

Hence, in general,

$$\lceil n \rceil = \lambda sz.\, s\lceil n-1 \rceil(s\lceil n-2 \rceil(\cdots(s\lceil 0 \rceil z)\cdots)).$$

Therefore $\lceil n \rceil$ contains exactly one copy of each of $\lceil 0 \rceil, \lceil 1 \rceil, \ldots, \lceil n-1 \rceil$, together with only linear overhead from the symbols s, applications, and the two abstractions. If a_n denotes the size of $\lceil n \rceil$, then

$$a_n = \sum_{k=0}^{n-1} a_k + O(n).$$

It follows that

$$a_{n+1} = 2a_n + O(1),$$

and hence

$$a_n = \Theta(2^n).$$

So the size of $\lceil n \rceil$ is $\Theta(2^n)$

(b)

- **rec** $\equiv \lambda xyz.zxy$
- **zero** $\equiv \lambda xy.y$
- **succ** $\equiv \lambda nxy.xn(\mathbf{rec}\ xyn)$

(c)

$$\mathbf{pred} \equiv \lambda x.\, x\,(\lambda sz.\, s)\,(\lambda sz.\, z).$$

This satisfies

$$\mathbf{pred}\,\lceil 0 \rceil = \lceil 0 \rceil \qquad \text{and} \qquad \mathbf{pred}\,\lceil n+1 \rceil = \lceil n \rceil.$$

Writing $K \equiv \lambda sz.\, s$, we have

$$\mathbf{pred}\,\lceil n+1 \rceil \to \lceil n+1 \rceil\, K\,\lceil 0 \rceil \to K\lceil n \rceil\big(K\lceil n-1 \rceil(\cdots(K\lceil 0 \rceil\lceil 0 \rceil)\cdots)\big).$$

Since each Kuv reduces to u in exactly 2 β-steps, the minimum number of reductions is obtained by reducing the outermost redex first, giving exactly 4 steps for $\mathbf{pred}\,\lceil n+1\rceil$. This is achieved by leftmost-outermost reduction.

The maximum number of reductions is obtained by reducing innermost redexes first, so that the entire discarded argument is evaluated. This takes $2(n+1)$ steps after the first 3 outer steps, hence exactly

$$2n+5$$

steps in total.

Thus:

$$\min = \Theta(1), \qquad \max = \Theta(n).$$

(d) The key observation for this part is that **rec** allows us to introduce primitive recursion. If we define

$$F \equiv \lambda n.\mathbf{rec}\ s\ z\ n,$$

then automatically we get

$$F\lceil 0\rceil = z, \quad F\lceil n+1\rceil = s\lceil n\rceil(F\lceil n\rceil).$$

We need to define two auxiliary terms first, i.e. **add** and **mult** corresponding to addition and multiplication operations with respect to the given encodings of natural numbers.

- $\mathbf{add} \equiv \lambda nm.\mathbf{rec}\ (\lambda kr.\mathbf{succ}\ r)\ m\ n$
- $\mathbf{mult} \equiv \lambda nm.\mathbf{rec}(\lambda kr.\mathbf{add}\ m\ r)\ \mathbf{zero}\ n$

We expect that **fac** satisfies the following:

$$\mathbf{fac}\lceil 0\rceil = \lceil 1\rceil, \quad \mathbf{fac}\lceil n+1\rceil = \mathbf{mult}\ \lceil n+1\rceil\ (\mathbf{fac}\ \lceil n\rceil)$$

Hence we can define

$$\mathbf{fac} \equiv \lambda n.\mathbf{rec}\ (\lambda kr.\mathbf{mult}\ (\mathbf{succ}\ k)\ r)\ \lceil 1\rceil\ n$$

For **fib** we need to additionally find a way to encode information about a pair of values:

- $\mathbf{pair} \equiv \lambda abf.f\ a\ b$
- $\mathbf{fst} \equiv \lambda p.p\ (\lambda ab.a)$
- $\mathbf{snd} \equiv \lambda p.p\ (\lambda ab.b)$

We need a term corresponding to a recursive procedure which computes the pair $(fib(n), fib(n+1))$ with base case $(fib(0), fib(1)) = (0,1)$ and step given by $(a,b) \mapsto (b, a+b)$. Define

$$\mathbf{fib} \equiv \lambda n.\mathbf{fst}\ (\mathbf{rec}\ (\lambda kp.\mathbf{pair}\ (\mathbf{snd}\ p)\ (\mathbf{add}\ (\mathbf{fst}\ p)\ (\mathbf{snd}\ p)))\ (\mathbf{pair}\ \lceil 0\rceil\ \lceil 1\rceil)\ n)$$

(e) The construction is somewhat similar to the one from part (b).

- $\mathbf{recList} \equiv \lambda nc\ell.\ell\ n\ c$
- $\mathbf{nil} \equiv \lambda nc.n$
- $\mathbf{cons} \equiv \lambda xpnc.c\ x\ p\ (p\ n\ c)$

(f)

- $\mathbf{length} \equiv \mathbf{recList}\,\lceil 0\rceil\,(\lambda x\,p\,r.\,\mathbf{succ}\,r)$
- $\mathbf{suffixes} \equiv \mathbf{recList}\,\mathbf{nil}\,(\lambda x\,p\,r.\,\mathbf{cons}\,p\,r)$

(g) For this part I will use some of the constructions provided in https://cs.ioc.ee/ewscs/2016/geuvers/geuvers-slides-lecture2.pdf. Let

$$\langle M_1, M_2, M_3\rangle \equiv \lambda z.z\ M_1\ M_2\ M_3$$

$$\mathbf{U_1^3} := \lambda xyz.x \qquad \mathbf{U_2^3} := \lambda xyz.y \qquad \mathbf{U_3^3} := \lambda xyz.z$$

Then define the three constructors by

$$\mathbf{var} := \lambda xe.e\ \mathbf{U_1^3}\ x\ e \qquad \mathbf{app} := \lambda pqe.e\ \mathbf{U_2^3}\ p\ q\ e \qquad \mathbf{lam} := \lambda pe.e\ \mathbf{U_3^3}\ p\ e$$

Now for the recursor, define

$$\mathbf{rec\Lambda} := \lambda valt.t\langle B_1, B_2, B_3\rangle \qquad \text{where}$$

$$B_1 := \lambda xz.v\ x \qquad B_2 := \lambda pqz.ap(pz)q(qz) \qquad B_3 := \lambda pz.lp(pz)$$

Question 2

(a) (i) Here Var is the variable rule, $\Rightarrow I$ is implication introduction, $\Rightarrow E$ is implication elimination (application), and $+E_1, +E_2$ are the two sum-elimination rules allowing one to view a term of type $A + B$ as having type A or type B, respectively.

$$\dfrac{\dfrac{\dfrac{\dfrac{(x : A + \big(A \Rightarrow (B \Rightarrow C)\big)) \in \{x : A + \big(A \Rightarrow (B \Rightarrow C)\big),\ y : B\}}{\{x : A + \big(A \Rightarrow (B \Rightarrow C)\big),\ y : B\} \vdash x : A + \big(A \Rightarrow (B \Rightarrow C)\big)}\ \textsc{Var}}{\{x : A + \big(A \Rightarrow (B \Rightarrow C)\big),\ y : B\} \vdash x : A \Rightarrow (B \Rightarrow C)}\ +E_2 \quad \dfrac{\dfrac{(x : A + \big(A \Rightarrow (B \Rightarrow C)\big)) \in \{x : A + \big(A \Rightarrow (B \Rightarrow C)\big),\ y : B\}}{\{x : A + \big(A \Rightarrow (B \Rightarrow C)\big),\ y : B\} \vdash x : A + \big(A \Rightarrow (B \Rightarrow C)\big)}\ \textsc{Var}}{\{x : A + \big(A \Rightarrow (B \Rightarrow C)\big),\ y : B\} \vdash x : A}\ +E_1}{\{x : A + \big(A \Rightarrow (B \Rightarrow C)\big),\ y : B\} \vdash xx : B \Rightarrow C}\ \Rightarrow E \quad \dfrac{(y : B) \in \{x : A + \big(A \Rightarrow (B \Rightarrow C)\big),\ y : B\}}{\{x : A + \big(A \Rightarrow (B \Rightarrow C)\big),\ y : B\} \vdash y : B}\ \textsc{Var}}{\{x : A + \big(A \Rightarrow (B \Rightarrow C)\big),\ y : B\} \vdash xxy : C}\ \Rightarrow E}{\{x : A + \big(A \Rightarrow (B \Rightarrow C)\big)\} \vdash \lambda y.\, xxy : B \Rightarrow C}\ \Rightarrow I}{\vdash \lambda x\, y.\, xxy : A + \big(A \Rightarrow (B \Rightarrow C)\big) \Rightarrow (B \Rightarrow C)}\ \Rightarrow I$$

(ii) We can take $A \equiv (B \Rightarrow B) \Rightarrow (B + (B \Rightarrow B)) \Rightarrow B$

$$\dfrac{\dfrac{\dfrac{\dfrac{\dfrac{\dfrac{(y : B + (B \Rightarrow B)) \in \{x : A + \big(A \Rightarrow (B \Rightarrow C)\big),\ y : B\}}{\{x : A + \big(A \Rightarrow (B \Rightarrow C)\big),\ y : B\} \vdash y : B + (B \Rightarrow B)}\ \textsc{Var}}{\{x : A + \big(A \Rightarrow (B \Rightarrow C)\big),\ y : B\} \vdash y : B \Rightarrow B}\ +E_2 \quad \dfrac{(x : B \Rightarrow B) \in \{x : A + \big(A \Rightarrow (B \Rightarrow C)\big),\ y : B\}}{\{x : A + \big(A \Rightarrow (B \Rightarrow C)\big),\ y : B\} \vdash x : B \Rightarrow B}\ \textsc{Var}}{\{x : A + \big(A \Rightarrow (B \Rightarrow C)\big),\ y : B\} \vdash xy : B}\ \Rightarrow E \quad \dfrac{(y : B + (B \Rightarrow B)) \in \{x : A + \big(A \Rightarrow (B \Rightarrow C)\big),\ y : B\}}{\{x : A + \big(A \Rightarrow (B \Rightarrow C)\big),\ y : B\} \vdash y : B + (B \Rightarrow B)}\ \textsc{Var}}{\{x : A + \big(A \Rightarrow (B \Rightarrow C)\big),\ y : B\} \vdash y : B}\ +E_1}{\{x : A + \big(A \Rightarrow (B \Rightarrow C)\big),\ y : B\} \vdash y(xy) : B}\ \Rightarrow E}{\{x : B \Rightarrow B\} \vdash \lambda y.\, y(xy) : (B + (B \Rightarrow B)) \Rightarrow B}\ \Rightarrow I}{\vdash \lambda xy.\, y(xy) : (B \Rightarrow B) \Rightarrow (B + (B \Rightarrow B)) \Rightarrow B}\ \Rightarrow I$$

(iii) Let $M \equiv \lambda x.xx$, and suppose for contradiction that

$$\vdash MM : A$$

for some type A. Then, by the application rule, there is a type S such that

$$\vdash M : S \qquad \text{and} \qquad \vdash M : S \Rightarrow A.$$

Choose such an S of minimal complexity.

Since $M = \lambda x.xx$, the derivation of $\vdash M : S \Rightarrow A$ must end with $\Rightarrow$-introduction, so

$$x : S \vdash xx : A.$$

For the application xx to be typable, x must be derivable both at some type T and at type $T \Rightarrow A$. Since the only additional typing rules are those for $+$, this means that S must contain T and $T \Rightarrow A$ among its $+$-components. Hence, from $\vdash M : S$, repeated $+$-elimination yields

$$\vdash M : T \qquad \text{and} \qquad \vdash M : T \Rightarrow A.$$

But T is strictly simpler than S, contradicting the minimality of S. Therefore Ω is not typable.

(b) We prove by induction on the shape of β-normal forms that every β-normal form is typable.

Recall that a term is in β-normal form iff it has the shape

$$s \equiv \lambda x_1 \cdots \lambda x_n.\, y\, s_1 \cdots s_m \qquad (n, m \geqslant 0),$$

where each s_i is itself in β-normal form.

We argue by induction on this form.

Base case: $s \equiv x$ is a variable. Choose any type A. Then, by the variable rule,

$$\{x : A\} \vdash x : A.$$

So every variable is typable.

Application spine case: $s \equiv y\, s_1 \cdots s_m$, where each s_i is in β-normal form. By the induction hypothesis, for each i there exist a context Γ_i and a type A_i such that

$$\Gamma_i \vdash s_i : A_i.$$

Choose any type B, and define

$$\Gamma \;=\; \{\, y : A_1 \Rightarrow \cdots \Rightarrow A_m \Rightarrow B \,\} \;+\; \Gamma_1 \;+\; \cdots \;+\; \Gamma_m.$$

By the assumed weakening property, for each i,

$$\Gamma \vdash s_i : A_i,$$

and also

$$\Gamma \vdash y : A_1 \Rightarrow \cdots \Rightarrow A_m \Rightarrow B.$$

Repeated applications of $\Rightarrow E$ therefore yield

$$\Gamma \vdash y\, s_1 \cdots s_m : B.$$

Hence $y\, s_1 \cdots s_m$ is typable.

Abstraction case: $s \equiv \lambda x.M$, where M is in β-normal form. By the induction hypothesis, there exist a context Θ and a type B such that

$$\Theta \vdash M : B.$$

If $x \in \mathit{Subjects}(\Theta)$, write $\Theta = \Gamma \cup \{x : A\}$. Then by $\Rightarrow I$,

$$\Gamma \vdash \lambda x.M : A \Rightarrow B.$$

If $x \notin \mathit{Subjects}(\Theta)$, choose any type A. By weakening,

$$\Theta + \{x : A\} \vdash M : B.$$

Since $x \notin \mathit{Subjects}(\Theta)$, we have $\Theta + \{x : A\} = \Theta \cup \{x : A\}$, so by $\Rightarrow I$,

$$\Theta \vdash \lambda x.M : A \Rightarrow B.$$

Thus $\lambda x.M$ is typable in either case.

Therefore every β-normal form is typable in this type system.

(c) We can use $(\lambda x.\mathbf{i})\mathbf{\Omega} \to_\beta \mathbf{i}$ as a counterexample. Observe that $\mathbf{i}$ is typable, e.g. $\vdash \mathbf{i} : A \Rightarrow A$. Suppose for contradiction that $\vdash (\lambda x.\mathbf{i})\mathbf{\Omega} : A \Rightarrow A$. Then the last rule must be application, so for some type C we would have

$$\vdash \lambda x.\mathbf{i} : C \Rightarrow (A \Rightarrow A) \qquad \text{and} \qquad \vdash \mathbf{\Omega} : C.$$

The first judgment is fine, but the second is impossible, since $\mathbf{\Omega}$ is not typable. Hence subject expansion fails.

(d) *Claim.* If $\Gamma \cup \{x : B\} \vdash s : A$ and $x \notin FV(s)$, then $\Gamma \vdash s : A$.

Proof. We argue by induction on the structure of s.

First note the following auxiliary fact: if $y \neq x$ and $\Gamma \cup \{x : B\} \vdash y : A$, then $\Gamma \vdash y : A$. Indeed, argue by induction on the derivation of $\Gamma \cup \{x : B\} \vdash y : A$. The last rule can only be Var, $+$-introduction, or one of the two $+$-eliminations, and each case is immediate.

Now for the main induction.

Case 1: $s \equiv y$. Since $x \notin FV(y)$, we have $y \neq x$, so the auxiliary fact gives $\Gamma \vdash y : A$.

Case 2: $s \equiv uv$. Assume $\Gamma \cup \{x : B\} \vdash uv : A$ and $x \notin FV(uv)$, so $x \notin FV(u), FV(v)$. By the Subject Construction Lemma,

$$A \equiv A_1 + \cdots + A_n$$

with, for each i, some B_i such that

$$\Gamma \cup \{x : B\} \vdash u : B_i \Rightarrow A_i \qquad \text{and} \qquad \Gamma \cup \{x : B\} \vdash v : B_i.$$

By the induction hypothesis, $\Gamma \vdash u : B_i \Rightarrow A_i$ and $\Gamma \vdash v : B_i$ for each i, hence $\Gamma \vdash uv : A_i$ for each i. Repeated $+$-introduction yields $\Gamma \vdash uv : A$.

Case 3: $s \equiv \lambda y.u$. Assume $\Gamma \cup \{x : B\} \vdash \lambda y.u : A$ and $x \notin FV(\lambda y.u)$. By α-renaming, we may assume $y \neq x$ and $y \notin \mathit{Subjects}(\Gamma)$; then $x \notin FV(u)$. By the Subject Construction Lemma,

$$A \equiv (B_1 \Rightarrow C_1) + \cdots + (B_n \Rightarrow C_n)$$

and for each i,

$$\Gamma \cup \{x : B\} \cup \{y : B_i\} \vdash u : C_i.$$

Since $y \neq x$ and $y \notin \mathit{Subjects}(\Gamma)$, this is $(\Gamma \cup \{y : B_i\}) \cup \{x : B\} \vdash u : C_i$. The induction hypothesis therefore gives $\Gamma \cup \{y : B_i\} \vdash u : C_i$, so by $\Rightarrow I$ we obtain $\Gamma \vdash \lambda y.u : B_i \Rightarrow C_i$ for each i. Repeated $+$-introduction now gives $\Gamma \vdash \lambda y.u : A$.

Thus in every case, if $\Gamma \cup \{x : B\} \vdash s : A$ and $x \notin FV(s)$, then $\Gamma \vdash s : A$. □

(e) We first prove an auxiliary lemma.

Lemma. Assume $x \notin \mathit{Subjects}(\Gamma)$. If $\Gamma \vdash u[v/x] : A$ and $\Gamma \vdash v : B_0$ for some B_0, then there is a type B such that

$$\Gamma \vdash v : B \qquad \text{and} \qquad \Gamma \cup \{x : B\} \vdash u : A.$$

Proof. By induction on the structure of u.

Case 1: $u \equiv y$.

If $y = x$, then $u[v/x] \equiv v$, so $\Gamma \vdash v : A$; take $B := A$. Since $(x : A) \in \Gamma \cup \{x : A\}$, we get $\Gamma \cup \{x : A\} \vdash x : A$.

If $y \neq x$, then $u[v/x] \equiv y$, so $\Gamma \vdash y : A$. Take $B := B_0$. By weakening, $\Gamma \cup \{x : B\} \vdash y : A$.

Case 2: $u \equiv pq$.

Then $u[v/x] \equiv p[v/x]\, q[v/x]$. By the Subject Construction Lemma,

$$A = A_1 + \cdots + A_n$$

with, for each i, some C_i such that

$$\Gamma \vdash p[v/x] : C_i \Rightarrow A_i \qquad \text{and} \qquad \Gamma \vdash q[v/x] : C_i.$$

By the induction hypothesis, for each i there are types B_i', B_i'' with

$$\Gamma \vdash v : B_i',\ \Gamma \cup \{x : B_i'\} \vdash p : C_i \Rightarrow A_i,$$

and

$$\Gamma \vdash v : B_i'',\ \Gamma \cup \{x : B_i''\} \vdash q : C_i.$$

Let $B_i := B_i' + B_i''$. Then $\Gamma \vdash v : B_i$, and by weakening

$$\Gamma \cup \{x : B_i\} \vdash p : C_i \Rightarrow A_i \qquad \text{and} \qquad \Gamma \cup \{x : B_i\} \vdash q : C_i,$$

so $\Gamma \cup \{x : B_i\} \vdash pq : A_i$.

Now let $B := B_1 + \cdots + B_n$. Then $\Gamma \vdash v : B$, and again by weakening

$$\Gamma \cup \{x : B\} \vdash pq : A_i \qquad (1 \leqslant i \leqslant n).$$

Hence repeated +-introduction yields $\Gamma \cup \{x : B\} \vdash pq : A$.

Case 3: $u \equiv \lambda y.r$.

By α-renaming, assume $y \neq x$, $y \notin FV(v)$, and $y \notin \mathit{Subjects}(\Gamma)$. Then $u[v/x] \equiv \lambda y.\, r[v/x]$. By the Subject Construction Lemma,

$$A = (C_1 \Rightarrow D_1) + \cdots + (C_n \Rightarrow D_n)$$

with

$$\Gamma \cup \{y : C_i\} \vdash r[v/x] : D_i \qquad (1 \leqslant i \leqslant n).$$

By weakening, $\Gamma \cup \{y : C_i\} \vdash v : B_0$. Applying the induction hypothesis in the context $\Gamma \cup \{y : C_i\}$, for each i there is B_i such that

$$\Gamma \cup \{y : C_i\} \vdash v : B_i \quad \text{and} \quad \Gamma \cup \{y : C_i\} \cup \{x : B_i\} \vdash r : D_i.$$

Since $y \notin FV(v)$, part (d) gives $\Gamma \vdash v : B_i$ for each i. Let $B := B_1 + \cdots + B_n$. Then $\Gamma \vdash v : B$, and by weakening

$$\Gamma \cup \{x : B\} \cup \{y : C_i\} \vdash r : D_i \qquad (1 \leqslant i \leqslant n).$$

Thus

$$\Gamma \cup \{x : B\} \vdash \lambda y.r : C_i \Rightarrow D_i \qquad (1 \leqslant i \leqslant n),$$

and repeated +-introduction yields $\Gamma \cup \{x : B\} \vdash \lambda y.r : A$. □

Corollary. Assume $x \notin \mathit{Subjects}(\Gamma)$. If $\Gamma \vdash u[v/x] : A$ and $\Gamma \vdash v : B_0$ for some B_0, then

$$\Gamma \vdash (\lambda x.u)\, v : A.$$

Proof. By the lemma, there is B such that $\Gamma \vdash v : B$ and $\Gamma \cup \{x : B\} \vdash u : A$. Hence $\Gamma \vdash \lambda x.u : B \Rightarrow A$, and therefore $\Gamma \vdash (\lambda x.u)\, v : A$. □

Proof of the main statement. Suppose

$$s = C[(\lambda x.u)v], \qquad t = C[u[v/x]], \qquad \Gamma \vdash t : A,$$

and also $\Gamma \vdash v : B_0$ for some B_0. By α-renaming, we may assume that $x \notin \mathit{Subjects}(\Gamma)$ and that x is fresh for the binders appearing in C.

We prove $\Gamma \vdash s : A$ by induction on the shape of the context $C[X]$.

Case 1: $C[X] \equiv X$. Then $t \equiv u[v/x]$ and $s \equiv (\lambda x.u)v$, so the claim is exactly the corollary.

Case 2: $C[X] \equiv C_0[X]\, w$.

Assume $\Gamma \vdash C_0[u[v/x]]\, w : A$. By the Subject Construction Lemma,

$$A = A_1 + \cdots + A_n$$

with, for each i, some D_i such that

$$\Gamma \vdash C_0[u[v/x]] : D_i \Rightarrow A_i \qquad \text{and} \qquad \Gamma \vdash w : D_i.$$

By the induction hypothesis,

$$\Gamma \vdash C_0[(\lambda x.u)v] : D_i \Rightarrow A_i \qquad (1 \leqslant i \leqslant n).$$

Hence $\Gamma \vdash C_0[(\lambda x.u)v]\, w : A_i$ for each i, and repeated $+$-introduction gives $\Gamma \vdash s : A$.

Case 3: $C[X] \equiv w\, C_0[X]$.

This is symmetric. From $\Gamma \vdash w\, C_0[u[v/x]] : A$, the Subject Construction Lemma yields

$$\Gamma \vdash w : D_i \Rightarrow A_i \qquad \text{and} \qquad \Gamma \vdash C_0[u[v/x]] : D_i$$

for suitable A_i, D_i. By the induction hypothesis,

$$\Gamma \vdash C_0[(\lambda x.u)v] : D_i,$$

hence $\Gamma \vdash w\, C_0[(\lambda x.u)v] : A_i$ for each i, and so $\Gamma \vdash s : A$.

Case 4: $C[X] \equiv \lambda y.\, C_0[X]$.

By α-renaming, assume $y \neq x$, $y \notin FV(v)$, and $y \notin \mathit{Subjects}(\Gamma)$. From

$$\Gamma \vdash \lambda y.\, C_0[u[v/x]] : A,$$

the Subject Construction Lemma gives

$$A = (D_1 \Rightarrow E_1) + \cdots + (D_n \Rightarrow E_n)$$

with

$$\Gamma \cup \{y : D_i\} \vdash C_0[u[v/x]] : E_i \qquad (1 \leqslant i \leqslant n).$$

By weakening, $\Gamma \cup \{y : D_i\} \vdash v : B_0$. Hence the induction hypothesis, applied in the context $\Gamma \cup \{y : D_i\}$, yields

$$\Gamma \cup \{y : D_i\} \vdash C_0[(\lambda x.u)v] : E_i \qquad (1 \leqslant i \leqslant n).$$

Therefore

$$\Gamma \vdash \lambda y.\, C_0[(\lambda x.u)v] : D_i \Rightarrow E_i \qquad (1 \leqslant i \leqslant n),$$

and repeated $+$-introduction gives $\Gamma \vdash s : A$. □

To remove the extra assumption $\Gamma \vdash v : B$, we restrict to the class of terms in which every abstraction $\lambda x.u$ satisfies $x \in FV(u)$.

If $s = C[(\lambda x.u)v] \to_\beta C[u[v/x]] = t$ and $x \in FV(u)$, then v occurs as a subterm of $u[v/x]$, hence as a subterm of t. Moreover, every subterm of a typable term is typable: this follows by a straightforward induction on subterms using the Subject Construction Lemma. Therefore, from $\Gamma \vdash t : A$ we obtain $\Gamma \vdash v : B$ for some B, and the first part applies. Hence such terms are closed under subject expansion.

The obstruction is exactly erasing redexes, where $x \notin FV(u)$; part (c) gives such a counterexample.

(f)

Proof. We prove that every strongly normalisable term is typable by induction on the following function

$$\mu(t) := (l(t), |t|),$$

where $l(t)$ is the sum of the lengths of all reduction sequences from t, and $|t|$ is the size of t.

If t is in β-normal form, then t is typable by part (b).

Now suppose t is strongly normalisable but not in normal form. Then

$$t = C[(\lambda x.u)v]$$

for some context $C[\ ]$, and contracting this redex yields

$$t' := C[u[v/x]].$$

Since $t \to_\beta t'$, every reduction sequence from t' gives one from t by prefixing the step $t \to_\beta t'$. Hence $l(t') < l(t)$, so $\mu(t') < \mu(t)$ (with respect to lexicographic ordering). As t' is strongly normalisable, the induction hypothesis gives a context Γ_0 and a type A such that

$$\Gamma_0 \vdash t' : A.$$

Next, v is a subterm of the strongly normalisable term t, hence is itself strongly normalisable. Also $|v| < |t|$, and every reduction sequence from v induces one from t by reducing inside that occurrence of v, so $l(v) \leqslant l(t)$. Therefore $\mu(v) < \mu(t)$. By the induction hypothesis, there are Γ_1 and B such that

$$\Gamma_1 \vdash v : B.$$

Let $\Gamma := \Gamma_0 + \Gamma_1$. By monotonicity of typing under context extension,

$$\Gamma \vdash t' : A \qquad \text{and} \qquad \Gamma \vdash v : B.$$

Since

$$t = C[(\lambda x.u)v] \qquad \text{and} \qquad t' = C[u[v/x]],$$

part (e) yields $\Gamma \vdash t : A$.

Thus t is typable. Hence every strongly normalisable term is typable. □

Question 3

A brief note on the approach: my initial solution to parts (a) and (b) turned out, upon comparison with [1], to be very close to the Scott–Mogensen encoding. To avoid merely reproducing that construction, I state only the essential definitions and results for those parts, and place the emphasis of the present solution on part (c).

Due to some technical difficulties with compilation of the file, I will use $\ulcorner - \urcorner$ as a quotation symbol.

(a) An injective self-encoding of Λ

We use a Scott–Mogensen style quotation, viewing Λ as the recursively generated syntax of variables, applications, and abstractions.

Definition of quotation. For each term $M \in \Lambda$, choose variables u, v, w fresh for M, and define $\ulcorner M \urcorner \in \Lambda$ recursively by

$$\begin{aligned} \ulcorner x \urcorner &:= \lambda u\, v\, w.\ u\, x, \\ \ulcorner P\, Q \urcorner &:= \lambda u\, v\, w.\ v\, \ulcorner P \urcorner\, \ulcorner Q \urcorner, \\ \ulcorner \lambda x.P \urcorner &:= \lambda u\, v\, w.\ w\, (\lambda x.\ulcorner P \urcorner). \end{aligned}$$

Thus a quoted term is a higher-order case analyser, with u, v, w corresponding to the constructors for variables, applications, and abstractions.

Basic properties. The map $M \mapsto \ulcorner M \urcorner$ is defined by structural recursion on M, hence is effective. It is also well defined up to α-equivalence, since different fresh choices of u, v, w yield α-equivalent codes.

Proposition 1. *The quotation map $\ulcorner - \urcorner : \Lambda \to \Lambda$ is injective modulo α-equivalence: if $\ulcorner M \urcorner \equiv_\alpha \ulcorner N \urcorner$, then $M \equiv_\alpha N$.*

The proof is by structural analysis of the three outer syntactic forms of a quoted term is given in Appendix A.

(b) A self-interpreter

Definition of the evaluator. Let $I := \lambda z.z$, and let Y be Curry's fixed-point combinator. Define

$$\begin{aligned} F &:= \lambda E.\lambda m.\ m\ I\ (\lambda p.\lambda q.\ (E\, p)(E\, q))\ (\lambda h.\lambda x.\ E(h\, x)), \\ e &:= Y F. \end{aligned}$$

Then e performs case analysis on a quoted term: it returns a variable unchanged, interprets both subterms of an application and applies the results, and reconstructs a λ-abstraction in the abstraction case.

Proposition 2. *For every $M \in \Lambda$, one has $e\, \ulcorner M \urcorner =_\beta M$. In particular, for every closed term s, $e\, \ulcorner s \urcorner =_\beta s$.*

The proof is by induction on the size of M and is given in Appendix B.

(c) Further operations on quoted terms

Besides the predicates **isvar**, **isapp**, and **isabs**, one can define further operations by case analysis on the three constructors. Fix the default code

$$d := \ulcorner I \urcorner, \qquad I := \lambda z.z,$$

used as a dummy output when the input does not have the required syntactic form.

Recognisers.

$$\begin{aligned}
\mathbf{isabs} &\equiv \lambda T.\, T\,(\lambda x.\mathbf{f})\,(\lambda p.\lambda q.\mathbf{f})\,(\lambda h.\mathbf{t}), & \mathbf{isabs}\,\ulcorner s\urcorner =_\beta \mathbf{t} &\iff s \equiv \lambda x.t,\\
\mathbf{isvar} &\equiv \lambda T.\, T\,(\lambda x.\mathbf{t})\,(\lambda p.\lambda q.\mathbf{f})\,(\lambda h.\mathbf{f}), & \mathbf{isvar}\,\ulcorner s\urcorner =_\beta \mathbf{t} &\iff s \equiv x,\\
\mathbf{isapp} &\equiv \lambda T.\, T\,(\lambda x.\mathbf{f})\,(\lambda p.\lambda q.\mathbf{t})\,(\lambda h.\mathbf{f}), & \mathbf{isapp}\,\ulcorner s\urcorner =_\beta \mathbf{t} &\iff s \equiv t\,u.
\end{aligned}$$

Projections for applications.

$$\begin{aligned}
\mathbf{left} &\equiv \lambda T.\, T\,(\lambda x.d)\,(\lambda p.\lambda q.p)\,(\lambda h.d),\\
\mathbf{right} &\equiv \lambda T.\, T\,(\lambda x.d)\,(\lambda p.\lambda q.q)\,(\lambda h.d).
\end{aligned}$$

Hence, if $s \equiv M\,N$, then

$$\mathbf{left}\,\ulcorner s\urcorner =_\beta \ulcorner M\urcorner, \qquad \mathbf{right}\,\ulcorner s\urcorner =_\beta \ulcorner N\urcorner.$$

Projection for abstractions.

$$\mathbf{bodyfun} \equiv \lambda T.\, T\,(\lambda x.d)\,(\lambda p.\lambda q.d)\,(\lambda h.h).$$

If $s \equiv \lambda x.M$, then

$$\mathbf{bodyfun}\,\ulcorner s\urcorner =_\beta \lambda x.\ulcorner M\urcorner,$$

so **bodyfun** extracts the function carrying the quoted body.

Constructors on codes.

$$\begin{aligned}
\mathbf{cvar} &\equiv \lambda x.\lambda v\,a\,\ell.\, v\,x,\\
\mathbf{capp} &\equiv \lambda p.\lambda q.\lambda v\,a\,\ell.\, a\,p\,q,\\
\mathbf{cabs} &\equiv \lambda h.\lambda v\,a\,\ell.\, \ell\,h.
\end{aligned}$$

Then

$$\mathbf{capp}\,\ulcorner M\urcorner\,\ulcorner N\urcorner =_\beta \ulcorner M\,N\urcorner, \qquad h = \lambda x.\ulcorner M\urcorner \implies \mathbf{cabs}\,h =_\beta \ulcorner \lambda x.M\urcorner.$$

Miscellaneous. A useful operation on codes of abstractions is *opening* the body at a variable:

$$\mathbf{freedom} \equiv \lambda A.\lambda y.\,(\mathbf{bodyfun}\ A)\,y.$$

If $A = \ulcorner \lambda x.M\urcorner$, then

$$\mathbf{bodyfun}\,A =_\beta \lambda x.\ulcorner M\urcorner,$$

and therefore, for every variable y,

$$\mathbf{freedom}\,\ulcorner \lambda x.M\urcorner\,y =_\beta (\lambda x.\ulcorner M\urcorner)\,y =_\beta \ulcorner M[y/x]\urcorner.$$

Thus **freedom** instantiates the bound variable of a quoted abstraction with the variable y, and hence performs capture-avoiding substitution at the level of codes.

A second useful operation is code-level η-expansion:

$$\mathbf{eta} \equiv \lambda p.\,\mathbf{cabs}\big(\lambda x.\,\mathbf{capp}\,p\,(\mathbf{cvar}\ x)\big).$$

Then, for every $M \in \Lambda$,

$$\mathbf{eta}\,\ulcorner M\urcorner =_\beta \ulcorner \lambda x.Mx\urcorner.$$

Hence **eta** uniformly adds one application of the bound variable to a quoted term. In particular, if $x \notin FV(M)$, then

$$\lambda x.Mx =_\eta M,$$

so $\mathbf{eta}\,\ulcorner M\urcorner$ is an η-expansion of $\ulcorner M\urcorner$.

Limitative results We end this section with one concrete limitation of the Scott-Mogensen encoding. A fairly natural question to ask is if one can define a test whether a quoted term is closed. For the present quotation, the answer is negative: there is no closed λ-term **closed** such that, for every $M \in \Lambda$,

$$\mathbf{closed}\ \ulcorner M \urcorner =_\beta \mathbf{t} \quad \Longleftrightarrow \quad M \text{ is closed.}$$

In the abstraction case,

$$\ulcorner \lambda x.M \urcorner = \lambda u\, v\, w.\, w(\lambda x.\ulcorner M \urcorner),$$

so the body is accessible only through

$$h = \lambda x.\ulcorner M \urcorner.$$

To inspect the body, one must apply h to some term b. For $\lambda x.x$ and $\lambda x.y$ (with y free), this gives

$$(\lambda x.\ulcorner x \urcorner)b \to_\beta \lambda u\, v\, w.\, u\, b, \qquad (\lambda x.\ulcorner y \urcorner)b \to_\beta \lambda u\, v\, w.\, u\, y.$$

Thus any closure test would have to distinguish these two variable cases: there would have to be a closed term V such that

$$V\, b =_\beta \mathbf{t}, \qquad V\, y =_\beta \mathbf{f}$$

for arbitrary y. But this is impossible, since V is closed: from $V\, y =_\beta \mathbf{f}$ one may substitute b for y, obtaining $V\, b =_\beta \mathbf{f}$, contradicting $V\, b =_\beta \mathbf{t}$.

Therefore no closed λ-term **closed** can decide whether a quoted term is closed. The obstruction is that, in this higher-order encoding, one cannot internally distinguish a genuinely free variable from one introduced when descending under an abstraction.

Conclusions

In this mini-project, we explored three distinct aspects of the λ-calculus.

The first question provided concrete examples of computation within the system and helped develop a more operational understanding of the formalism. Working through explicit reductions made the computational content of the calculus particularly transparent.

The second question illustrated how the typing discipline can be extended beyond the simply typed λ-calculus, and how such extensions affect the structure and expressive power of the system. This highlighted the close relationship between typing rules and the behaviour of terms.

The third question, by its more open-ended nature, invited an exploration of different methods of self-encoding. This part was especially instructive, as it revealed a concrete instance of the trade-off between generality, simplicity, and internal manipulability in formal systems. My initial inclination, influenced by a background in logic, was to pursue a Gödel-numbering approach. However, once it became clear that this would make the later parts of the problem significantly more cumbersome, I turned instead to a higher-order encoding. That choice led to a cleaner evaluator and more direct code-level operations, but at the cost of certain limitations, as discussed at the end of Question 3.

Overall, the project showed how the λ-calculus can be studied simultaneously as a model of computation, as a typed formal system, and as an object language capable of representing its own syntax. I find this interplay between computation, structure, and self-reference particularly interesting.

Appendix A. Proof of injectivity of the quotation

We first consider two elementary lemmas about the quotation map.

Lemma 1. *For every $M \in \Lambda$,*

$$FV(\ulcorner M \urcorner) = FV(M).$$

Proof. By induction on the structure of M.

If $M \equiv x$, then

$$\ulcorner x \urcorner = \lambda u\, v\, w.\, u\, x,$$

so $FV(\ulcorner x \urcorner) = \{x\} = FV(x)$.

If $M \equiv P\, Q$, then

$$\ulcorner P\, Q \urcorner = \lambda u\, v\, w.\, v\, \ulcorner P \urcorner\, \ulcorner Q \urcorner,$$

hence

$$FV(\ulcorner P\, Q \urcorner) = FV(\ulcorner P \urcorner) \cup FV(\ulcorner Q \urcorner) = FV(P) \cup FV(Q) = FV(P\, Q),$$

using the induction hypothesis.

If $M \equiv \lambda x.P$, then

$$\ulcorner \lambda x.P \urcorner = \lambda u\, v\, w.\, w\, (\lambda x.\ulcorner P \urcorner),$$

so

$$FV(\ulcorner \lambda x.P \urcorner) = FV(\lambda x.\ulcorner P \urcorner) = FV(\ulcorner P \urcorner) \backslash \{x\} = FV(P) \backslash \{x\} = FV(\lambda x.P),$$

again by the induction hypothesis. □

Lemma 2. *Let x, z be variables, and assume $z \notin FV(M)$. Then*

$$\ulcorner M[z/x] \urcorner \equiv_\alpha \ulcorner M \urcorner [z/x].$$

Proof. By induction on the structure of M. Before carrying out the induction, we may α-rename bound variables of M so that none of them is equal to z, and so that the auxiliary variables used in the quotation clauses are chosen fresh for x, z, and the relevant subterms. This avoids any possibility of capture.

If $M \equiv y$ is a variable, then there are two subcases.

If $y = x$, then

$$\ulcorner M[z/x] \urcorner = \ulcorner z \urcorner = \lambda u\, v\, w.\, u\, z$$

and

$$\ulcorner M \urcorner [z/x] = (\lambda u\, v\, w.\, u\, x)[z/x] = \lambda u\, v\, w.\, u\, z.$$

If $y \neq x$, then $M[z/x] \equiv y$, and similarly

$$\ulcorner M[z/x] \urcorner = \lambda u\, v\, w.\, u\, y = (\lambda u\, v\, w.\, u\, y)[z/x] = \ulcorner M \urcorner [z/x].$$

If $M \equiv P\, Q$, then

$$\ulcorner (P\, Q)[z/x] \urcorner = \ulcorner P[z/x]\ Q[z/x] \urcorner = \lambda u\, v\, w.\, v\, \ulcorner P[z/x] \urcorner\, \ulcorner Q[z/x] \urcorner.$$

By the induction hypothesis,

$$\ulcorner P[z/x] \urcorner \equiv_\alpha \ulcorner P \urcorner [z/x], \qquad \ulcorner Q[z/x] \urcorner \equiv_\alpha \ulcorner Q \urcorner [z/x].$$

Therefore

$$\ulcorner (P\, Q)[z/x] \urcorner \equiv_\alpha \lambda u\, v\, w.\, v\, (\ulcorner P \urcorner [z/x])\, (\ulcorner Q \urcorner [z/x]) = \ulcorner P\, Q \urcorner [z/x].$$

If $M \equiv \lambda y.P$, then, after the preliminary α-renaming just mentioned, we may assume $y \neq z$. Since $z \notin FV(\lambda y.P)$, we also have $z \notin FV(P)\backslash\{y\}$, and capture-avoiding substitution gives

$$(\lambda y.P)[z/x] \equiv \lambda y.(P[z/x]).$$

Hence

$$\ulcorner(\lambda y.P)[z/x]\urcorner = \ulcorner\lambda y.(P[z/x])\urcorner = \lambda u\, v\, w.\, w\,(\lambda y.\ulcorner P[z/x]\urcorner).$$

By the induction hypothesis,

$$\ulcorner P[z/x]\urcorner \equiv_\alpha \ulcorner P\urcorner[z/x].$$

Thus

$$\ulcorner(\lambda y.P)[z/x]\urcorner \equiv_\alpha \lambda u\, v\, w.\, w\,(\lambda y.(\ulcorner P\urcorner[z/x])).$$

Because $y \neq z$, the substitution $[z/x]$ does not affect the binder y, so

$$\lambda u\, v\, w.\, w\,(\lambda y.(\ulcorner P\urcorner[z/x])) = (\lambda u\, v\, w.\, w\,(\lambda y.\ulcorner P\urcorner))[z/x] = \ulcorner\lambda y.P\urcorner[z/x].$$

This completes the induction. □

Proof. We prove the following statement by induction on the structure of M:

$$\textit{For every } N \in \Lambda, \textit{ if } \ulcorner M\urcorner \equiv_\alpha \ulcorner N\urcorner, \textit{ then } M \equiv_\alpha N.$$

Recall that every quoted term has, up to α-equivalence, one of the three shapes

$$\lambda u\, v\, w.\, u\, x, \qquad \lambda u\, v\, w.\, v\, P\, Q, \qquad \lambda u\, v\, w.\, w\, H,$$

according as the original term is a variable, an application, or an abstraction. Since α-conversion does not change the outer syntactic constructor, if

$$\ulcorner M\urcorner \equiv_\alpha \ulcorner N\urcorner,$$

then M and N must have the same outer syntactic form.

Case 1: $M \equiv x$ **is a variable.** Then N must also be a variable, say $N \equiv y$. Thus

$$\lambda u\, v\, w.\, u\, x \equiv_\alpha \lambda u'\, v'\, w'.\, u'\, y.$$

Choose fresh variables a, b, c not occurring free in either side. This is possible by Lemma 1, since

$$FV(\ulcorner x\urcorner) = \{x\}, \qquad FV(\ulcorner y\urcorner) = \{y\}.$$

By α-conversion we may therefore rewrite both terms with the same outer binders:

$$\lambda a\, b\, c.\, a\, x \equiv_\alpha \lambda a\, b\, c.\, a\, y.$$

Hence

$$a\, x \equiv_\alpha a\, y.$$

Now α-conversion does not alter free variables, so necessarily $x = y$. Therefore $M \equiv_\alpha N$.

Case 2: $M \equiv P\, Q$ **is an application.** Then N must also be an application, say $N \equiv R\, S$. We have

$$\lambda u\, v\, w.\, v\, \ulcorner P\urcorner\, \ulcorner Q\urcorner \equiv_\alpha \lambda u'\, v'\, w'.\, v'\, \ulcorner R\urcorner\, \ulcorner S\urcorner.$$

Choose fresh variables a, b, c not occurring free in either side; again this is justified by Lemma 1. After α-conversion of the outer binders,

$$\lambda a\, b\, c.\, b\, \ulcorner P\urcorner\, \ulcorner Q\urcorner \equiv_\alpha \lambda a\, b\, c.\, b\, \ulcorner R\urcorner\, \ulcorner S\urcorner.$$

Therefore

$$b\,\ulcorner P\urcorner\,\ulcorner Q\urcorner \equiv_\alpha b\,\ulcorner R\urcorner\,\ulcorner S\urcorner.$$

Since the head variable b is free and the application bracketing is fixed, it follows that

$$\ulcorner P\urcorner \equiv_\alpha \ulcorner R\urcorner, \qquad \ulcorner Q\urcorner \equiv_\alpha \ulcorner S\urcorner.$$

By the induction hypothesis,

$$P \equiv_\alpha R, \qquad Q \equiv_\alpha S.$$

Hence

$$M \equiv P\,Q \equiv_\alpha R\,S \equiv N.$$

Case 3: $M \equiv \lambda x.P$ **is an abstraction.** Then N must also be an abstraction, say $N \equiv \lambda y.Q$. Thus

$$\lambda u\,v\,w.\,w\,(\lambda x.\ulcorner P\urcorner) \equiv_\alpha \lambda u'\,v'\,w'.\,w'\,(\lambda y.\ulcorner Q\urcorner).$$

Choose fresh variables a, b, c not occurring free in either side, and rename the outer binders on both sides accordingly. We obtain

$$\lambda a\,b\,c.\,c\,(\lambda x.\ulcorner P\urcorner) \equiv_\alpha \lambda a\,b\,c.\,c\,(\lambda y.\ulcorner Q\urcorner).$$

Hence

$$c\,(\lambda x.\ulcorner P\urcorner) \equiv_\alpha c\,(\lambda y.\ulcorner Q\urcorner),$$

and therefore

$$\lambda x.\ulcorner P\urcorner \equiv_\alpha \lambda y.\ulcorner Q\urcorner.$$

Now choose a variable z fresh for both $\lambda x.P$ and $\lambda y.Q$. A standard property of α-equivalence of abstractions yields

$$\ulcorner P\urcorner[z/x] \equiv_\alpha \ulcorner Q\urcorner[z/y].$$

By Lemma 2,

$$\ulcorner P[z/x]\urcorner \equiv_\alpha \ulcorner P\urcorner[z/x], \qquad \ulcorner Q[z/y]\urcorner \equiv_\alpha \ulcorner Q\urcorner[z/y].$$

Therefore

$$\ulcorner P[z/x]\urcorner \equiv_\alpha \ulcorner Q[z/y]\urcorner.$$

Applying the induction hypothesis to the strictly smaller terms $P[z/x]$ and $Q[z/y]$, we get

$$P[z/x] \equiv_\alpha Q[z/y].$$

Hence

$$\lambda x.P \equiv_\alpha \lambda z.P[z/x] \equiv_\alpha \lambda z.Q[z/y] \equiv_\alpha \lambda y.Q.$$

Thus $M \equiv_\alpha N$.

All cases are complete, so the quotation map is injective modulo α-equivalence. □

Appendix B. Correctness of the evaluator

The following argument follows [1] closely, adapted to the present notation and quotation convention. We first isolate the substitution behaviour built into the quotation of abstractions.

Lemma 3. *Let $P \in \Lambda$, and let x, y be variables. Then*

$$(\lambda x.\ulcorner P\urcorner)\, y =_\beta \ulcorner P[y/x]\urcorner.$$

Proof. By induction on the structure of P. Throughout, we work modulo α-equivalence, so bound variables may be renamed fresh when needed.

Case 1: $P \equiv x$**.** Then

$$(\lambda x.\ulcorner x\urcorner)\, y = (\lambda x.\lambda u\, v\, w.\, u\, x)\, y \to_\beta \lambda u\, v\, w.\, u\, y = \ulcorner y\urcorner = \ulcorner x[y/x]\urcorner.$$

Case 2: $P \equiv z$ **with** $z \neq x$**.** Then

$$(\lambda x.\ulcorner z\urcorner)\, y = (\lambda x.\lambda u\, v\, w.\, u\, z)\, y \to_\beta \lambda u\, v\, w.\, u\, z = \ulcorner z\urcorner = \ulcorner z[y/x]\urcorner.$$

Case 3: $P \equiv M\, N$**.** Then

$$\begin{aligned}
(\lambda x.\ulcorner M\, N\urcorner)\, y &= (\lambda x.\lambda u\, v\, w.\, v\, \ulcorner M\urcorner\, \ulcorner N\urcorner)\, y \\
&\to_\beta \lambda u\, v\, w.\, v\, (\ulcorner M\urcorner[y/x])\, (\ulcorner N\urcorner[y/x]) \\
&=_\beta \lambda u\, v\, w.\, v\, \ulcorner M[y/x]\urcorner\, \ulcorner N[y/x]\urcorner \qquad \text{(by the induction hypothesis)} \\
&= \ulcorner M[y/x]\ N[y/x]\urcorner \\
&= \ulcorner (M\, N)[y/x]\urcorner.
\end{aligned}$$

Case 4: $P \equiv \lambda z.M$**.** By the variable convention, we may assume $z \neq x$ and $z \neq y$. Then

$$\begin{aligned}
(\lambda x.\ulcorner \lambda z.M\urcorner)\, y &= (\lambda x.\lambda u\, v\, w.\, w(\lambda z.\ulcorner M\urcorner))\, y \\
&\to_\beta \lambda u\, v\, w.\, w\big(\lambda z.\ulcorner M\urcorner[y/x]\big) \\
&=_\beta \lambda u\, v\, w.\, w\big(\lambda z.\ulcorner M[y/x]\urcorner\big) \qquad \text{(by the induction hypothesis)} \\
&= \ulcorner \lambda z.M[y/x]\urcorner \\
&= \ulcorner (\lambda z.M)[y/x]\urcorner.
\end{aligned}$$

This completes the induction. □

Proof of proposition 2. Let

$$A \equiv \lambda pq.\, (e\, p)(e\, q), \qquad B \equiv \lambda hy.\, e(h\, y).$$

Recall that $e = Y\, F$, where

$$F := \lambda E.\lambda m.\ m\ I\ (\lambda p.\lambda q.\ (E\, p)(E\, q))\ (\lambda h.\lambda x.\ E(h\, x)).$$

Thus $e =_\beta F\, e$.

We prove, by induction on the size $|M|$ of the term M, that

$$e\, \ulcorner M\urcorner =_\beta M.$$

Case 1: $M \equiv x$ **is a variable.** Then

$$\begin{aligned}
e\, \ulcorner x\urcorner &=_\beta (F\, e)\, \ulcorner x\urcorner \\
&=_\beta \ulcorner x\urcorner\, I\, A\, B \\
&= (\lambda u\, v\, w.\, u\, x)\, I\, A\, B \\
&\to_\beta I\, x \\
&\to_\beta x.
\end{aligned}$$

Case 2: $M \equiv P\,Q$ **is an application.** Then

$$\begin{aligned} e\,\ulcorner P\,Q\urcorner &=_\beta (F\,e)\,\ulcorner P\,Q\urcorner \\ &=_\beta \ulcorner P\,Q\urcorner\, I\,A\,B \\ &= (\lambda u\,v\,w.\,v\,\ulcorner P\urcorner\,\ulcorner Q\urcorner)\,I\,A\,B \\ &\to_\beta A\,\ulcorner P\urcorner\,\ulcorner Q\urcorner \\ &\to_\beta (e\,\ulcorner P\urcorner)(e\,\ulcorner Q\urcorner) \\ &=_\beta P\,Q, \end{aligned}$$

where the last step uses the induction hypothesis, since $|P|, |Q| < |P\,Q|$.

Case 3: $M \equiv \lambda x.P$ **is an abstraction.** Choose a variable y fresh for $\lambda x.P$. Then

$$\begin{aligned} e\,\ulcorner\lambda x.P\urcorner &=_\beta (F\,e)\,\ulcorner\lambda x.P\urcorner \\ &=_\beta \ulcorner\lambda x.P\urcorner\, I\,A\,B \\ &= (\lambda u\,v\,w.\,w(\lambda x.\ulcorner P\urcorner))\,I\,A\,B \\ &\to_\beta B(\lambda x.\ulcorner P\urcorner) \\ &\to_\beta \lambda y.\,e\big((\lambda x.\ulcorner P\urcorner)\,y\big) \\ &=_\beta \lambda y.\,e\,\ulcorner P[y/x]\urcorner \qquad \text{by Lemma 3} \\ &=_\beta \lambda y.\,P[y/x] \qquad \text{by the induction hypothesis, since } |P[y/x]| = |P| < |\lambda x.P| \\ &\equiv_\alpha \lambda x.P. \end{aligned}$$

Since terms are identified modulo α-equivalence, this proves

$$e\,\ulcorner\lambda x.P\urcorner =_\beta \lambda x.P.$$

All cases are proved. Therefore, for every $M \in \Lambda$,

$$e\,\ulcorner M\urcorner =_\beta M.$$

In particular, for every closed term s,

$$e\,\ulcorner s\urcorner =_\beta s.$$

□

References

[1] Torbenn Æ. Mogensen, *Efficient self-interpretation in lambda calculus* J. Functional Programming 2 (3): 345-364, July 1992 Cambridge University Press

[2] Andrew D. Ker, *Lambda Calculus and Types* Oxford University Computing Laboratory, ver. 01/2024